开源软件的价值

曼努埃尔·霍夫曼
哈佛商学院

弗兰克·纳格尔
哈佛商学院

周亚诺
多伦多大学

工作文件 24-038

版权所有 © 2024 曼努埃尔·霍夫曼、弗兰克·纳格尔和周亚诺。

工作论文目前仍处于草案阶段。本工作论文仅用于征求意见和讨论之目的，未经版权所有者许可不得复制。工作论文副本可向作者索取。

作者衷心感谢Linux基金会提供的资金与行政支持，若无此支持，人口普查数据将无法获取。我们高度赞赏哈佛商学院研究计算服务部、哈佛创新科学实验室、Linux基金会以及软件成分分析数据提供方Snyk、Synopsys网络安全研究中心和fossa所提供的支持。感谢Tianli Li和Misha Bouzinier提供的卓越研究协助。同时感谢软件开发者Boris Martinovic，以及微软的Rich Lander和Scott Hanselman对.NET生态系统的深刻见解。我们还收到了来自哈佛商学院价值观与估值会议、哈佛商学院D3研究日及2023年管理学院大会参与者的宝贵反馈。

本研究的部分资金由哈佛商学院提供。

开源软件的价值

曼努埃尔·霍夫曼[a]　　弗兰克·纳格尔[b]　　周亚诺[c]

此版本：2024年1月1日

摘要

非货币性（免费）产品的价值本质上难以评估。开源软件（OSS）就是一个典型的例子——这种全球性的公共产品在经济中发挥着关键作用，也是当今大多数技术的基础。然而，由于开源软件的非货币属性以及缺乏集中的使用追踪数据，其价值难以量化，因此在经济指标中基本未被纳入考量。尽管先前研究已估算出重新开发这类软件的供给侧成本，但数据匮乏阻碍了对开源软件所创造的更大规模需求侧（使用）价值的评估。为此，为全面理解广泛使用的开源软件的经济与社会价值，我们利用来自两个互补来源的独特全球数据，这些数据记录了全球数百万企业的开源软件使用情况。首先，我们通过计算重新开发最广泛使用的开源软件所需的成本来估算供给侧价值；随后，基于每家使用该软件的企业若开源软件不存在时需自行开发软件所产生的替代价值，计算出需求侧价值。我们估算出广泛使用的开源软件的供给侧价值为40亿美元。虽然总价值为150亿美元，但需求侧价值更为可观，达到8.8万亿美元。研究发现，如果没有开源软件（OSS），企业用于软件的研发支出将比当前高出3.5倍。我们样本中排名前六的编程语言贡献了开源软件需求侧价值的84%；此外，仅5%的开源软件开发者就创造了96%的需求侧价值。

*JEL 分类：*H4；O3；J0
*关键词：*开源软件，全球公共产品

致谢：作者衷心感谢Linux基金会提供的资金与行政支持，若无此支持，人口普查数据将无法获取。我们高度赞赏哈佛商学院研究计算服务部、哈佛创新科学实验室、Linux基金会以及软件成分分析数据提供方Snyk、Synopsys网络安全研究中心和fossa的支持。感谢Tianli Li和Misha Bouzinier提供的卓越研究协助。同时感谢软件开发者Boris Martinovic，以及微软的Rich Lander和Scott Hanselman对.NET生态系统的深刻见解。我们还收到了哈佛商学院价值观与估值会议、哈佛商学院D [3]研究日及2023年管理学院会议参与者的宝贵反馈。

[a] 哈佛商学院，哈佛大学，地址：马萨诸塞州奥尔斯顿市西大道150号6.220室，邮编：02134；电子邮件：mhoffmann@hbs.edu。
[b] 哈佛商学院，哈佛大学，摩根213室，Soldiers Field，波士顿，马萨诸塞州，02134，电子邮件：fnagle@hbs.edu。
[c] 多伦多大学罗特曼管理学院，地址：加拿大安大略省多伦多市圣乔治街105号，邮编：M5S 3E6；电子邮件：yanuo.zhou@rotman.utoronto.ca。

1. 引言

2011年，风险投资家马克·安德森曾提出著名论断："软件正在吞噬整个世界"（Andreessen，2011），这一观点如今几乎无人质疑。近期，风险投资家约瑟夫·杰克斯更指出："开源软件吞噬软件的速度，比软件吞噬世界的速度还要快"（Jacks，2022）。其他最新研究也得出了相似结论：开源软件（OSS）——其源代码公开供他人检查、使用和修改，通常以去中心化方式开发并免费分发——已存在于96%的代码库中（Synopsys，2023）；部分商业软件中高达99.9%的代码均为开源软件（Musseau 等，2022）。尽管开源软件早期常借鉴现有专有软件的功能，但如今它已涵盖人工智能（AI）、量子计算、大数据及数据分析等多个领域的尖端技术。然而，尽管开源软件对所有软件（进而对整个经济）的重要性日益凸显，其实际影响仍难以准确衡量。传统上，为了衡量某种商品或服务所创造的价值，经济学家会将价格（p）乘以销售数量（q）。然而，在开源软件（OSS）中，p通常为零，因为源代码是公开可用的；而q则因对代码复制和重用方式的限制有限而难以确定。例如，如果一家公司从公共代码仓库下载了一段开源软件代码，它可能在内部（合法地）复制数千次，然后与供应商或客户共享（同样合法），因此公开的下载数据并不充分。尽管近期一些研究试图估算p的价值（下文将讨论），但用于估算q的数据要么缺失，要么仅适用于少数几个开源软件包，难以处理。本文利用来自多个来源的新收集数据，旨在同时估算p和q，并借此阐明一个核心问题：开源软件的价值究竟有多大？

理解开源软件（OSS）的价值至关重要，这不仅源于其在经济中的重要作用，更因为它堪称数百年来"公地"这一古老经济概念最成功且最具影响力的现代实践案例——而这一概念本身正面临着所谓的"公地悲剧"。该概念可追溯至公元前4世纪哲学家亚里士多德的论述："为大多数人所共有之物，往往得到最少的照料；人们最关注属于自己的东西，而对公共财产则漠不关心。"（Aristotle，1981）威廉·福斯特·劳埃德（1833）在现代经济思想中重新提出了这一观点，并以用于牲畜放牧的共享土地为例进行了阐释。

众多牧牛人各自都有过度利用公共资源的动机。加勒特·哈丁（1968）在撰写题为《公地悲剧》的文章时，将这一概念引入了更广泛的学术视野。在此基础上，埃莉诺·奥斯特罗姆因其研究荣获诺贝尔奖——她的研究揭示了通过社区协作避免“公地悲剧”的途径，而无需依赖政府强制立法来管理和保护公共资源（Ostrom 1990）。共享放牧地与共享数字基础设施之间的相似性显而易见：公共牧场可供放牧牲畜、进而养活人类，这对农业经济至关重要；而无需重复编写他人已编写的代码，正是现代经济的关键所在。此外，在这两种情境下，尽管人们深知草地和代码都是经济的重要投入要素，但衡量其实际价值却十分困难。正如著名数学家兼物理学家开尔文勋爵所言：“无法测量的，就无法改进。”“如果你无法改进并维护这些资源，那么这些公共产品可能会因自身成功而崩溃——它们被过度使用，却缺乏足够的投资（Lifshitz-Assaf and Nagle，2021）。因此，衡量开源软件（OSS）创造的价值，对于数字经济及其上建立的整个经济体系的未来健康发展至关重要。”

重要的是，近期研究虽试图解决这些测量问题，但仍未能全面反映开源软件（OSS）使用的广度与深度——这正是本文旨在填补的空白。例如，研究人员通过采用创新方法估算现有开源软件的劳动力替代价值：2019年美国开发的开源软件价值达380亿美元（Robbins 等人，2021），欧盟开发的开源软件价值为10亿欧元（Blind 等人，2021），具体方法是估算重写现有开源软件所需的人力成本。这些研究在估算若所有现有开源软件明日消失所需替代成本方面成效显著。然而，所得估算结果基于两个关键假设：首先，所有开源软件在使用价值上具有同等重要性；其次，开源软件的概念依然存在，社会只需重写一次代码即可满足需求——这解决了前述p价值缺失的问题，却未能解决q价值缺失的问题。在一个完全不存在开源软件（OSS）的世界里，每款开源软件无需仅由单一机构重写，而需由所有使用该软件的企业共同重写（前提是这些企业能够在内部自由共享该软件）。其他研究（Greenstein and Nagle，2014； MurcianoGoroff，Zhuo，and Greenstein，2021）对此假设进行了更深入探讨，尽管其研究范围较为有限。

这两项研究均仅聚焦于Web服务器（这类服务器面向互联网公众开放，因此易于量化）。通过采用不同方法，两项研究均针对同一类软件测量了q指标，并基于企业提供的闭源替代品价格，运用商品替代价值法推算了p。利用美国数据得出的估算结果显示：2012年开源软件Apache Web Server的价值为20亿美元（Greenstein and Nagle，2014）；而到2018年，Apache与日益流行的开源Web服务器nginx的总价值已达45亿美元（Murciano-Goroff et al.，2021）。尽管Web服务器是开源软件生态系统的重要组成部分，但其占比却很小。我们旨在在此前研究的重要成果基础上，既拓宽研究维度又深化分析深度，以构建更全面的开源软件价值评估体系。

为了从广度和深度两方面评估开源软件（OSS）的价值，我们采用了两个主要数据来源，从而深入了解全球数万家企业的开源软件使用情况。第一个来源是“免费及开源软件应用库第二次普查”（Nagle 等人，2022）。该普查项目通过与多家软件成分分析（SCA）公司合作，编制了各类最广泛使用的开源软件清单。SCA公司负责扫描企业代码库，确保其未违反任何开源软件许可协议，并同步追踪客户及产品所使用的全部开源代码。普查项目整合了多家SCA公司的数据，基于数百万个开源软件使用观测数据，构建了涵盖数万家企业的开源软件使用数据集。第二个数据来源是BuiltWith数据集，我们利用其对近九百万个网站的扫描结果，识别这些网站所采用的基础技术（包括开源软件库）。BuiltWith数据已被多项学术研究采用（DeStefano & Timmis 2023；Dushnitsky & Stroube 2021；Koning等人）。（2022年），但据我们所知，这是首个专注于开源软件（OSS）使用情况的研究。Census II数据与BuiltWith数据具有互补性：前者侧重于企业销售软件中内置的开源软件，后者则关注企业网站中集成的开源软件，从而降低了跨数据集重复计算观测值的可能性。总体而言，这两个数据集的结合构成了迄今为止最完整的开源软件使用情况（q）测量指标。此外，通过聚焦于企业广泛部署和使用的开源软件，而非考虑开源软件仓库中的所有项目，我们改进了先前研究的方法论——降低了因某些项目虽标榜为公开可用开源软件却未实际投入实际使用而导致测量误差的可能性。若未考虑这一点

测量误差会导致对OSS实际价值的高估，因为广泛使用的项目将与完全未使用的项目采用相同的估值方法。

为估算p，我们遵循上述文献方法，采用劳动力替代价值进行计算。首先，通过统计特定开源软件包中的代码行数，并应用构建成本模型II（Boehm，1984； Boehm 等，2009，亦称 COCOMO II）来估算从头编写该代码所需的人工小时数，从而计算出单个企业重新开发该软件包所需的劳动力成本。随后，利用Salary Expert提供的全球薪资数据，准确估算出若该开源软件不存在时企业需承担的劳动力成本。这些成本可与前述开源软件包层面的q值相结合，最终得出供需双方所有开源软件的综合价值。

如果我们作为一个社会决定在供应端重新开发所有广泛使用的开源软件（OSS），我们得出的价值范围在12.2亿至62.2亿美元之间。然而，考虑到开源软件的实际使用情况，若每家使用开源软件包的企业都必须从零开始重新开发（例如，当时根本不存在开源软件这一概念），其需求端价值将高出数个数量级，范围在2.59万亿美元至13.18万亿美元之间。我们发现开源软件的价值存在显著差异：不同编程语言、内部开发与对外发布的编程工作之间价值分布各异。此外，程序员对价值贡献也存在巨大差异——仅5%的程序员就贡献了供应端和需求端超过90%的价值。目前我们使用的数据可以说是衡量企业使用开源软件所创造价值最全面的资料来源。不过，与任何研究项目一样，现有证据并不完整；我们认为实际价值被低估了，因为我们的数据未包含操作系统——这是开源软件中一个被大幅遗漏的重要类别。

本研究对学术界、实践工作者及政策制定者作出了四项重要贡献。首先，它提供了迄今为止最全面的开源软件（OSS）价值评估——不仅考量了开源软件的供给端因素（开发成本），还以前所未有的规模分析了使用/需求端的影响。以往对开源软件价值的评估要么过于笼统（仅估算大量开源软件的供给端成本），要么过于单一（仅评估某一特定类型开源软件的价值）；而本研究通过运用独特的数据集，同时实现了这两点，从而更深入地理解了开源软件应用的广度与深度。此外，本研究并未衡量所有开源软件的价值，而是专注于企业实际使用的开源软件所产生的价值。

本研究旨在明确开源软件（OSS）在产品开发与网站构建中的实际应用，从而减少那些无法准确统计生产过程中实际使用开源软件类型的研究中产生的测量误差。该成果基于并拓展了多项重要研究（如Blind等人，2021；Greenstein与Nagle，2014；Murciano-Goroff等人，2021；Robbins等人，2021），这些研究致力于揭示这一对现代经济贡献巨大的关键资源的价值——尽管其实际贡献难以量化。通过这项研究，我们为关于信息技术对生产率影响的长期学术讨论（Brynjolfsson，1993； Brynjolfsson 与 Hitt，1996； Nagle，2019 a； Solow，1987）——即所谓的“生产率悖论”（即IT投资对生产率统计数据的影响有限）——提供了新的见解。这一争论已延伸至人工智能发展的新兴领域（Brynjolfsson、Rock与Syverson，2018）。我们的研究通过揭示开源软件存在的巨大社会层面成本节约效应（进而提升生产率），为这一学术讨论作出了重要贡献。

其次，我们的研究通过揭示与开源软件（OSS）投资相关的新型数据来源，为无形资本研究带来了方法论上的突破。先前研究表明，无形资本在经济增长（Corrado、Hulten和Sichel，2009）及企业价值（Peters 和 Taylor，2017）中日益发挥重要作用，但其往往未被有效量化或存在误判现象（Eisfeldt 和 Papanikolaou，2014）。此外，我们展示了如何利用这些数据来源来准确理解企业实际进行的软件投资，以及若无开源软件存在时企业本应进行的投资规模。这一发现具有重要价值，因为软件投资已成为推动创新（Branstetter、Drev和Kwon，2019）与企业绩效（Krishnan 等，2000）的关键无形资本类型。

第三，我们的研究结果有助于帮助企业与管理者认识到开源软件（OSS）对其生产活动的重要性，并有力支持了以下观点：开源软件用户不应仅享受免费资源，还应积极参与开源软件的创建与维护（例如Henkel，2008； Nagle，2018）。此类贡献仅占企业若无开源软件所需成本的一小部分，而用户积极参与维护所使用的开源软件，对开源生态系统的发展与未来可持续性至关重要（Lifshitz-Assaf and Nagle，2021； Zhang et al.，2019）。

第四，也是最后一点，我们的研究有助于为政策制定者提供参考——这些政策制定者近期已开始认识到开源软件（OSS）对经济日益增长的重要性，并采取了支持开源生态系统的相关措施（欧盟委员会，2020年；第14028号行政命令，2021年）。然而，这些努力大多集中在保护现有的开源生态系统上，这一点固然至关重要，但.....

我们的研究虽未深入支持创建新的开源软件（OSS），但其成果有助于揭示开源软件对整体经济的重要性，并为呼吁社会加强对这一关键资源的支持提供了有力依据。研究还表明，开源软件创造的大部分价值实际上源自少数贡献者。尽管长期以来人们已认识到少数开源贡献者承担了主要工作，但我们获得了新的见解：对于广泛使用的开源项目而言，这一现象尤为显著；社会对这些贡献者的支持对于开源软件乃至整个经济的未来成功至关重要。

本文其余部分结构安排如下：第2节阐述了实证研究背景与数据；第3节讨论了研究方法，包括测量过程中遇到的挑战；第4节对开源软件的价值进行了评估；第5节为全文总结。

2. 经验背景与数据

尽管自由开源软件的概念自20世纪50年代便已存在，但直到20世纪80年代，得益于理查德·斯托曼及其发起的GNU项目和自由软件基金会（Maracke，2019）的努力，这一理念才真正普及开来。然而，直到20世纪90年代，随着林纳斯·托瓦兹发布了Linux内核——这一如今已被广泛采用的开源操作系统（Tozzi，2016），开源软件才真正迎来蓬勃发展。如今，开源软件被视为数字经济的关键基石，被软件开发者广泛应用于从手机、汽车、冰箱到尖端人工智能等各个领域（Lifshitz-Assaf and Nagle，2021）。

我们采用两个互补的主要数据源来评估开源软件（OSS）的价值。第一个数据源是面向内部的“人口普查”数据集，它有助于我们识别企业开发的产品中所使用的开源软件代码；第二个数据集是面向外部的“BuiltWith”数据集，它使我们能够识别消费者通过企业网站直接使用的开源软件代码。由于这两个数据集中的原始数据仅包含软件包名称、版本号和包管理器名称，而不包含任何源代码相关信息，我们首先获取了每个软件包的公开代码仓库，其中包含软件包级别的详细信息，如代码行数及所使用的编程语言。有人可能会担心使用两个独立数据集会导致价值计算出现重复计算的情况。然而，“人口普查”与“BuiltWith”数据集的样本重叠度极低：两个数据集中仅存在18个共同的软件包。此外，重复计算几乎不会构成问题，因为这两个数据集反映了开源软件使用的不同维度：“人口普查”数据集记录的是面向内部使用的打包软件，而“BuiltWith”数据集则记录了面向外部网站的软件使用情况。最后，作为补充.....

在数据集构建过程中，我们采用了GHTorrent——这是GitHub（全球最受欢迎的开源软件托管平台）上所有开源软件相关活动的详细历史记录，也是开源软件研究领域广泛使用的数据来源（例如：Burton等人，2017；Conti、Peukert和Roche，2023；Fackler、Hofmann和Laurentsyeva，2023；Kim，2020；Tang、Wang和Tong，2023）。这些详尽的历史数据使我们能够更深入地理解开源软件价值的创造机制，具体表现为更清晰地掌握各开源开发者贡献的分布情况。下文将详细阐述这三个数据源的具体内容及其数据处理方法。

Census。《自由与 Open Source 软件 The Census II》（以下简称Census）由 Linux Foundation 与哈佛大学 Innovation Science 实验室联合完成（Nagle 等人，2022）。[1] Census 的数据来源于三家主要软件构成分析（SCA）公司的数据汇总，这些公司在全球拥有数千家客户。SCA公司受委托扫描客户的代码库，收集其专有软件中嵌入的开源软件使用情况，以确保不违反任何开源软件许可协议。[2]这一过程通常作为并购交易尽职调查的一部分。与公开来源提供的其他开源软件需求指标（如软件包下载量和代码变更次数）不同，该数据的收集方式确保我们能够准确掌握企业的内部开源软件使用规模与类型。此外，该方法还能追踪每个软件包所依赖的组件，从而揭示通常隐藏且难以获取的间接开源软件使用情况。[3]最终结果超过2。2020日历年（2020年1月1日至12月31日）期间，对SCA客户公司开发的产品中使用的OSS软件包进行了700万次观测。[4]

人口普查项目基于图书馆的命名系统.io对软件包名称进行了标准化。

– 这是一个由 Tidelift 维护的广泛使用的网站，收录了超过八百万个开源软件包的相关信息。本次统计重点分析了使用量排名前2000的软件包。

[1] 数据收集与汇总的具体方法详见 Nagle 等人（2022）发布的普查报告。

[2] 开源软件（OSS）的使用许可差异显著：部分许可极为宽松，允许代码以任何形式重用——包括用于商业用途的专有代码；而另一些许可则严格限制重用，要求最终生成的代码必须遵循相同的开源软件许可协议（即“左版”许可）。关于开源软件许可的详细论述可参阅 Lerner 与 Tirole（2005）及 Almeida 等人（2017）的研究成果。

[3] 间接使用的开源软件（OSS）需通过依赖关系分析来捕捉，这对于准确评估企业所依赖的开源软件的整体范围至关重要。例如，如果企业的专有代码调用了开源软件包A，而该软件包A又调用了软件包B，那么仅查看直接调用关系就会忽略软件包B是企业专有代码不可或缺的构建模块这一事实。

[4] 《人口普查》将OSS软件包定义为“可由包管理器安装和管理的软件单元”，并将包管理器定义为“能够自动化软件包安装及管理流程的软件”（Nagle 等人，2022）。

本研究基于三家最知名的SCA供应商提供的数据，旨在识别使用最广泛的开源软件（OSS），而非关注使用分布中的长尾部分。因此，使用记录不足五次的软件包被排除在外。由于用JavaScript编程语言编写、通常托管在Node Package Manager（NPM）上的软件包通常体积更小（代码行数更少），因此其使用频率往往更高（因为开发者需要集成大量小型软件包而非少数大型软件包）。为此，本次统计专门选取了 NPM 平台上使用量最高的1000个软件包，以及非 NPM 托管平台上使用量最高的1000个软件包。

该最终数据集涵盖了人口普查原始数据中观测到的开源软件（OSS）总使用量的70%。[5]对于人口普查中的这2,000个OSS包，我们识别了其在GitHub（开源软件最广泛使用的托管平台）上维护的原始代码。[6]我们首先尝试通过libraries.io获取每个包的GitHub仓库统一资源定位符（URL），通过这种方法成功将1,657个包与仓库匹配。[7]对于没有对应仓库的URL，我们参照Singh（2020）的方法进行谷歌搜索：具体而言，针对每个未匹配的包，使用Google API搜索包名和“GitHub Repository”，并将结果中的首个GitHub仓库URL视为最佳匹配地址。[8]这使得另有174个包成功匹配到仓库。最后，我们手动检索了剩余169个包的URL以确定其对应仓库。整个过程最终使人口普查中的2,000个包中有1,840个成功匹配到包含原始源代码的代码仓库。经确认，这些独特的软件包要么已从 GitHub 下载页面移除，要么已转为专有软件（因此原始源代码不再可用），因此通过手动搜索，我们得以将其剔除。

[5] 在长尾使用分布中，构成完整人口普查数据剩余30%的各类数据包未被纳入最终报告，因此无法被纳入我们的分析。

[6] GitHub是一个托管与协作平台，贡献者可通过该平台协调开源软件（OSS）项目的开发与发布。自2008年成立以来，GitHub已发展成为全球最大的开源软件开发枢纽。截至2023年1月，GitHub拥有超过3.7亿个代码仓库和1亿多名开发者。除个人用户外，众多私营企业也积极使用该平台，其中包括微软（2018年收购GitHub）、Facebook、Google以及众多其他大小企业。

[7] 我们尝试访问所有作为完整性检查而获取的仓库URL，以确认其功能正常。对于在GitHub上无法访问的仓库，我们手动找到了正确的URL。

[8] 作为稳健性验证，我们随机选取了50条通过Google搜索方法获得的软件包存储库匹配结果，并进行了人工核查。所有匹配结果均经人工确认无误，这为自动化匹配方法提供了额外的支持依据。

这160个未匹配的样本包（占人口普查2000人样本量的不足8%）经我们的分析证实具有高度可靠性。

BuiltWith数据库。该数据库收录了全球所有公共网站的扫描数据，并识别了它们所使用的技术。与侧重于开源软件（OSS）使用的内向性人口普查数据不同，BuiltWith数据对企业和机构网站中专有软件与开源软件的使用情况进行了统一扫描，未作明确区分。为在BuiltWith中区分开源软件与专有软件，我们选取了技术类别中包含“JavaScript及其库”的开源网页开发软件子集——该子集对应人口普查数据中的 NPM OSS类别，共生成778条观测记录。这一抽样选择基于两个原因：首先，该分类由BuiltWith自行构建，我们将其作为开源软件的替代指标，以区别于商业软件；其次，JavaScript作为构建网站的核心技术之一，在GitHub上的使用量位居全球首位（GitHub，2022），因此能从需求侧视角精准捕捉最重要的开源软件。扫描数据涵盖2020年1月1日至11月16日期间880万个独立网站及7280万条对应的开源软件使用观测记录。此外，为确保准确衡量私营部门所使用的开源软件（OSS）的实际价值，我们将其采用的JavaScript相关开源软件领域（数据来源于BuiltWith平台），与Orbis、Compustat和PitchBook三大主流企业活动数据库中收录的企业网站进行了匹配。这些数据库均收录了全球注册企业的信息。通过这种匹配机制，我们有效排除了非商业网站（例如个人网站）所使用的开源软件对分析结果的影响。最终匹配率达到38.6%，对应约340万家不同企业的网站数据。

对于 BuiltWith 数据，我们无法采用最初用于人口普查的方法（即使用 libraries.io 来辅助识别仓库 URL），因为 BuiltWith 仅提供了与软件包相关联的技术名称，而未包含其他信息（例如软件包名称及包管理器名称）。[9]因此，我们首先采用前述的谷歌搜索方法，成功匹配出 695 个软件包对应的仓库。随后，我们手动检索了剩余 83 个未匹配软件包的 GitHub URL，最终新增了 46 个软件包。总体而言，在 778 个软件包中，我们成功识别出 741 个。

[9] 由于软件包及其管理器名称缺失，无法通过libraries.io实现精确匹配。技术名称是面向客户的产品名称，其技术性和精确度通常低于内部开发用途的软件包名称。因此，在 libraries.io 搜索中使用技术名称可能会导致严重的歧义。

数据仓库匹配结果。与人口普查数据类似，其余37个未匹配的数据包（占比不足5%）已被排除在分析之外，因为这些数据包已从GitHub上删除。

GHTorrent数据库。为获取OSS价值创造的分散度指标，我们采用了GHTorrent数据库——该数据库通过GitHub的表征状态传输（REST）应用程序编程接口（API）收录了GitHub上的全部活动历史记录。我们利用其存储的GitHub仓库信息、开发者提交记录及开发者公开资料来评估每位开发者的贡献度。在开发者贡献度分析中，我们通过两步缩小样本范围：首先，根据我们联合Census-BuiltWith样本中的仓库URL，从GHTorrent中筛选出相应的GitHub仓库及其提交记录[10]；其次，通过剔除约8000名（占12%）被认定为机器人的GitHub贡献者，聚焦于人类对OSS的实际贡献[11]。最终样本包含约6万名开发者及230万次提交记录。

为了准备这三个数据集用于估算，我们首先使用开源软件包 pygount（用于统计代码行数）和 linguist（用于识别编程语言）来确定每个软件包的代码行数及所使用的编程语言。[12]我们将每种不同的编程语言归入三个不同的类别，从更可能由人类编写到更可能由机器编写（见表 A1）。类别 1 包含编程语言和标记语言（最可能由人类编写），类别 3 包含数据（最可能由机器编写），类别 2 则包含介于两者之间的内容，例如配置文件和批处理程序（这些内容有时由人类编写，有时由机器编写，但仅凭代码难以判断）。[13]在主要估算中，我们仅使用类别 1；同时在附录中对类别 2 和 3 进行了稳健性验证，因此我们的结果代表了一个下限。最后，针对部分分析，我们深入展示了排名前五的编程语言（根据 GitHub 2022 年发布的 2020 年数据进行分类）。排名前五的编程语言包括 C（含 C# 和 C++）、Java 和 JavaScript。

---

[10] 匹配率超过96%，未匹配的代码库仅占我们下文计算得出的开源软件（OSS）需求价值的0.15%。

[11] 该过滤机制基于GHTorrent对“虚假用户”的分类标准，同时还会识别包含“bot”或“robot”等关键词且被特殊字符包围的用户名。相较于GitHub创新图谱（https://innovationgraph.github.com/）等采用月度提交频率阈值进行机器人检测的方法，此方案更为保守。

[12] https://pypi.org/project/pygount/ 和https://github.com/github-linguist/linguist

[13] 需要注意的是，我们的关注点在于重新编写由人类而非机器人编写的开源软件（OSS）代码所产生的成本。然而，在此情况下直接从机器人账户中移除所有开源贡献是不可行的，因为我们无法查看GitHub上机器人账户所编写的具体代码行。

Python 和 TypeScript。我们还将 Go 加入了这一顶级编程语言列表，因为 Go 在开源软件（OSS）领域正得到越来越广泛的应用。

3. 方法论

我们首先采用劳动力市场方法，从OSS的供给端和需求端两个维度来衡量其价值。[14]该理论假设是：我们所处的世界中并不存在OSS，每家企业在使用特定OSS时都必须重新创造该技术。通过劳动力市场方法，我们计算出每个OSS方案的劳动力替代成本。为估算各方案的价值，我们首先在单个方案层面应用 COCOMO II模型（Boehm，1984； Boehm 等，2009），随后对所有方案价值进行汇总，得出供给端的劳动力市场替代价值。接着，我们将供给端价值除以各企业使用每个方案的次数，并剔除企业内部的重复使用情况，从而得出需求端价值。

其次，我们超越总体层面，深入剖析价值创造过程中的不平等现象。在开源软件（OSS）领域，与许多创造性工作类似，通常只有少数人贡献了大部分成果，而其他人则贡献较小份额（这一现象常被称为“80/20法则”，即80%的工作由20%的人完成）。研究表明，这些高频贡献者往往因其努力而在OSS社区中获得重要影响力（Hanisch等人，2018）。因此，为了更好地理解开发者在价值创造中的贡献分布情况，我们首先利用GHTorrent数据，通过两种方式识别每位开发者的贡献：a）直接评估其对OSS的价值贡献；b）统计其参与贡献的仓库总数。随后，我们检验这两种贡献衡量指标的集中程度，以明确究竟是大量还是少数开发者贡献了我们所衡量的总价值。下文将详细阐述劳动力市场分析方法及估值不平等的具体细节。

3.1 劳动力市场方法

对于劳动力市场方法，我们通过计算一个软件包的替代价值来估算开源软件的价值。具体而言，我们需要计算雇佣一名程序员并支付其具有市场竞争力的工资来重现该软件包所需的成本。为了估算这一供给侧价值（V Lab%r！），我们会汇总上述讨论的所有开源软件包，然后统计每个独特软件包中的代码行数。

[14] 在另一种版本中，我们采用了劳动力市场与商品市场相结合的方法，该方法更贴近 Greenstein 和 Nagle（2014）以及Murciano-Goroff等人（2021）的研究框架。然而，由于数据限制，将此方法应用于我们的研究情境需要做出诸多额外假设。为简化表述，我们将其称为“商品市场方法”，相关细节、统计数据及局限性均列于附录A。

数据包。[15]对于每个独特的数据包i，我们先计算其价值，然后对所有这些价值求和以获得总价值：

$$V_S^{劳动力}=\sum_{i=1}^{N} V_{S_i} = \sum_{i=1}^{N} P_i * 1 . \quad (1)$$

在本计算中，我们隐含地未纳入任何生产外部性，因为我们假设不同软件包之间不存在会降低编程成本的知识溢出效应。[16]该方法与其他研究估算开源软件（OSS）供给侧成本的方法类似（Blind 等人，2021； Nagle ，2019b； Robbins 等人，2021）。

随后，我们通过整合每个套餐的使用信息（Q），计算出OSS的需求侧价值：

$$V_D^{Labor} = \sum_{i=1}^{N} V_{D_i} = \sum_{i=1}^{N} P_i * Q_i . \quad (2)$$

在此，我们未纳入消费外部性因素，即当某个软件包被创建时，我们不允许普通公众从中获得任何收益；同时我们进一步确保每家企业仅替换其自身使用过一次的软件包，因为被替换的软件包可在企业内部作为俱乐部商品使用（例如参见 Cornes 和 Sandler，1996）。价值$V_S^{劳动}$反映了重新编写所有广泛使用的开源软件（OSS）一次所需的成本（例如：OSS的概念依然存在，但所有这些软件包都需要从头开始重写）；而价值VLaborD则反映了每家使用这些开源软件包的企业为支付开发者重写这些软件包所产生的成本（例如：OSS本身已不复存在）。

对于供给和需求模型，我们均通过构建式成本模型II（简称 COCOMO II，Boehm 1984，Boehm et al. 2009）计算每个软件包（pi）的美元价值。该模型此前已被美国国防部用于估算软件项目成本，并在多项研究中用于评估开源软件的价值（Blind et al.，2021；Nagle ，2019b； Robbins et al.，2021）。该模型具有高度灵活性，可实现代码行向美元价值的非线性转换，其建模方程如下：

[15] 我们仅使用纯代码行，排除了文档注释行和空行。因此，我们低估了重新编写每个软件包的真实价值。
[16] 与代表性代理模型类似（例如参见 Williamson 2006），我们可以认为每个软件包均由不同的程序员编写，这些程序员彼此完全相同（因此技能水平相同，但不会通过学习而提高效率）。

$$E_i = \alpha\eta L_i^{\beta} \quad (3)$$

其中L代表代码行数（单位：千行），E代表每个项目i的人力投入（单位：人月）。与 Blind 等人（2021）的研究一致，我们采用 α 、 β 和 η 的默认参数值。参数 α 和 β 分别为非线性调整因子，设定值为2.94和1.0997；参数 η 是人力投入调整因子，可根据对产品、硬件、人员及项目特性的主观评估进行调整。由于缺乏各项目的先验信息，我们将 η 设为默认值1。为计算每个开源软件项目的价格（pi），我们将公式(3)的计算结果乘以程序员在公开市场上获得的加权全球平均薪资。全球平均薪资的计算参考了《 Salary Expert》报告中2021年GitHub开发者数量排名前30个国家软件开发者的基准月薪数据（Wachs 等人，2022）。[17]各国权重为其活跃GitHub贡献者数量占该国总贡献者总数的比例。我们进一步利用低工资水平（印度）和高工资水平（美国）的劳动力市场来设定工资范围，以便更准确地理解：在重构整个开源软件（OSS）的过程中，所使用的程序员群体不同会导致价值产生何种差异。[18]

3.2 贡献度衡量

为了更深入地理解价值是如何被创造的，以及这种创造是否均衡或不均衡，我们分三个步骤构建了可视化图表。第一步，我们衡量了开发者对价值的贡献；第二步，我们统计了开发者参与贡献的代码仓库数量；最后，我们运用广为人知的洛伦兹曲线概念，对这两项数据进行了可视化呈现，以便更清晰地了解贡献不平等的程度。具体细节如下所述。

价值贡献。我们计算了每位开发者对开源软件（OSS）所做出的供需价值贡献。在仓库层面，我们通过计算每位开发者提交的代码量占仓库总提交量的比例，量化了其工作贡献比例；随后分别将该比例与仓库的需求值和供给值相乘，从而得出具体数值。

[17] Wachs 等人（2022）的研究共涵盖179个县，但排名前30的国家集中了全球超过88%的活跃贡献者，其余国家的占比均低于0.6%。前30个国家详见表A2。

[18] 我们选择高薪和低薪参考国家时，综合考虑了GitHub活跃开发者数量与软件开发者群体的平均年薪水平。

该独立贡献者对代码库的增值贡献。最后，我们汇总每位开发者在所有代码库中的价值贡献。来自唯一开发者Dev、V_j^{Dev}的个体价值贡献可表示为：

$$V_j^{开发} = \sum_{Ni} \sigma_i^{开发} * V_{ij} \tag{7}$$

其中σ_i^{Dev}表示焦点开发者在仓库i中提交的代码占比；V_{ij}是公式(1)和(2)中定义的整个仓库i的需求或供给价值（其中j∈{D，S}）；N表示我们主样本中的仓库数量，即Census与BuiltWith两个样本的合并数据。

代码库贡献量。该指标表示特定开发者所贡献的代码库数量，具体计算方式如下：

$$N^{Dev} = \sum_i^N \mathbb{1}\{\sigma_i^{Dev} > 0\} \tag{8}$$

其中1表示指示函数，当开发者向仓库i提交了非零数量的代码变更时，该函数值为1。这一指标反映了各开发者所承担的开源软件需求多样性。结合价值贡献度指标，它们有助于我们判断整体产生的价值是否集中在少数开发者手中。从开源软件生态系统的整体发展及其多样性角度来看，开发者参与多个仓库而非仅限于少数几个，通常更为理想。

测量贡献值的离散程度。为直观分析开发者贡献值的离散情况，我们采用了洛伦兹曲线（Lorenz，1905），分别针对需求侧和供给侧数值进行展示。洛伦兹曲线是衡量不平等现象的成熟方法，因此能帮助我们更清晰地理解私营经济中开发者对开源软件（OSS）贡献的离散程度。如公式(7)所示，开发者根据其对OSS需求和供给的贡献值被系统地按升序排列。随后，这些排名被标准化为0至100百分位数的范围，作为洛伦兹曲线的横坐标；纵坐标则显示相应的价值贡献值V_j^{Dev}。结果部分的图形展示将阐明开发者间价值贡献的不平等程度。为补充分析，我们

我们进一步通过绘制方程(8)来探究代码仓库贡献度 N^{Dev}的分布情况。这使我们能够确定：显著的价值不平等现象究竟是源于顶级贡献者主要专注于少数极受欢迎的代码仓库，还是源于他们参与了更广泛的成功代码仓库。

4. 结果

通过应用 COCOMO II的劳动力市场方法，我们获得了OSS价值的全球估算值。计算总体价值时，首先需要基础代码行数（用于计算供给端价值），随后需获取使用统计数据（用于计算需求端价值）。鉴于不同编程语言的价值可能存在显著差异，如上所述，我们还列出了2020年调查期间的主要编程语言。

--- 表1如下 ---

表1分别展示了两个数据集的描述性统计结果。面向内部使用的Census软件包（图A）包含超过2.617亿行代码，其中72%的代码来自主流编程语言。其平均软件包包含14.2万行代码，而主流语言子集的平均代码量略低，为11.3万行。从使用频率来看，Census软件包被调用超过270万次，其中92%的调用量来自主流语言；平均每个软件包被调用1,472.4次，而主流语言的调用量更高，约为1,497.5次。面向外部使用的BuiltWith数据也呈现类似趋势，但具体水平有所不同：BuiltWith包含的软件包代码总量超过8,200万行，其中71%来自主流编程语言；该样本中平均每个软件包包含11.1万行代码，而主流语言的平均代码量较低，约为8万行。这是因为BuiltWith数据主要由基于JavaScript的软件包构成，这类软件包通常比其他语言编写的软件包更精简。BuiltWith软件包的使用次数超过14.2万次，其中99.97%的使用量来自主流编程语言。随后，我们利用这些原始数据，通过劳动力市场方法计算了所有开源软件的价值，并估算了供需双方共同创造的价值。

--- 表2如下 ---

表2展示了基于企业相关的Census和BuiltWith联合样本对开源软件（OSS）价值的估算结果。表2中的所有估算数据均仅基于表A1中归入第一类的软件语言——这类语言最有可能由人类而非机器编写。[19]第一列分别列出了采用低收入国家（印度）工资水平、全球平均工资水平以及高收入国家（美国）工资水平的估算结果（如前所述）。若要使用印度开发者平均工资水平来重新开发所有广泛使用的开源软件（即：OSS的概念依然存在，但需删除当前所有代码并从头开始编码），所需投资为12.2亿美元；相比之下，若采用美国开发者平均工资水平，则需62.2亿美元才能完成相同工作。而采用基于上述地理分布对开源软件贡献度加权的全球程序员资源池时，所需投资金额介于高低收入国家之间，约为41.5亿美元。将这些数据与同类研究结果进行对比，有助于理解对所有开源软件（OSS）的估值（先前研究）与广泛使用的开源软件（本研究）之间的差异。Robbins 等人（2021）和 Blind 等人（2021）采用与我们类似的方法，估算出2019年美国产生的OSS价值为380亿美元，2018年欧盟产生的OSS价值为10亿欧元。Wachs 等人（2022）指出，约50%的OSS贡献来自美国和欧盟的合计贡献。综合来看，这些研究得出的全球OSS总价值为780亿美元。因此，我们针对仅限于面向企业且广泛使用的OSS所提出的41.5亿美元供应侧价值中值，是一个可信的下限估计值；该数值凸显了在未考虑特定OSS包是否被广泛使用的情况下，对OSS整体供应侧价值的更高估值。这进一步表明，最广泛使用的OSS其供应侧价值约占所有OSS供应侧价值的5.5%。

表2的第二列包含了基于劳动力市场方法得出的需求侧估算数据。研究发现，如果企业必须重新开发所有使用的开源软件（OSS）包（例如，OSS本身已不复存在，每家使用过OSS包的企业都需重新开发），那么仅使用低工资国家或高工资国家的劳动力，相关总成本将分别介于2.59万亿美元至13.18万亿美元之间。而由全球程序员组成的团队重新开发所有广泛使用的OSS，所需成本约为8.80万亿美元。解读这一数字时需注意：

[19] 附录中的表格展示了当包含软件语言分类桶1和2（表A3）以及全部三个软件语言分类桶（表A4）时，表2中的等效数值。

这一计算方法略显复杂，但仍具有可行性。根据Statista（2023年）的报告，2020年（与我们的数据年份相同）全球软件收入为5317亿美元。然而，这代表的是软件的流量而非存量。基于政府估算——软件在三年内完全折旧——我们可以进行粗略估算：将2020年使用的全部预装软件采购价值视为2018至2020年间销售总额（1.54万亿美元）。此外，该数据仅涵盖预装软件的支出，不包括定制采购或内部开发的软件。关于私营部门软件总体投资的最佳估算数据来自美国国民收入与生产账户（NIPA 2023）。2020年国民账户数据显示，美国私营企业软件支出达4792亿美元，其中45%（2155亿美元）为预装软件。若假设全球其他地区也采用相同比例，则2020年企业用于软件的实际支出总额为3.4万亿美元（= 1.54万亿美元/0.45）。将这一粗略估算与基于全球平均工资（8.8万亿美元）得出的开源软件价值需求侧估算相结合，表明企业若需聘请内部开发者编写当前免费使用的开源软件，将支出12.2万亿美元（=3.4万亿美元 + 8.8万亿美元），相当于其当前支出的三倍半。

--- 图1 此处 ---

图1展示了各大主流编程语言开源软件（OSS）价值的差异性。图A呈现了供应端价值，其中劳动价值以纵轴显示。研究发现，用Go语言开发的OSS包价值最高，达8.03亿美元——若这些OSS包不存在，开发者需从零开始开发同等价值的软件。紧随其后的是JavaScript和Java，价值分别为7.58亿美元和6.58亿美元。C和 TypeScript的价值分别为4.06亿美元和3.17亿美元，而Python在主流语言中价值最低，仅约5500万美元。JavaScript不仅是自2014年以来GitHub上的主导语言（GitHub，2022），也是我们数据中价值最高的语言之一。相比之下，Python的受欢迎程度持续上升，从第四位跃升至2020年GitHub所有OSS包使用的第二大语言，但在我们的主流语言排名中仍位列末位。

图B展示了主要编程语言在需求端的价值表现。根据实际使用产生的价值计算，Go语言的价值是紧随其后的JavaScript语言的四倍以上。

TypeScript（一种基于JavaScript扩展的语言）发展势头迅猛，从2017年全球十大编程语言中的第十位跃升至2020年的第四位；这一趋势在我们的数据中也得到印证——TypeScript已成为需求侧第三重要的编程语言。紧随这两种Web语言之后的是C语言，而Java和Python则远远落后。

--- 图2 正位于此处 ---

图2根据我们的内向型（Census）和外向型（BuiltWith）数据源，分别展示了各语言的OSS价值估算结果。图A和图B重点分析了Census数据中的供需侧估算。尽管JavaScript的影响显著较低，但其影响模式与整合两种数据源后的总体趋势（图1）基本一致。在图A所示的Census供给侧数据中，Java的价值排名第二，而Census中的JavaScript代码对整体价值的贡献明显较低。同样，C、Python和 TypeScript的供给侧价值主要由Census数据驱动。从图B的需求侧数据可以看出，Go是面向内码开发最主流的语言，而其他语言在相对占比上几乎可以忽略不计。

图2的C面板和D面板展示了BuiltWith数据集的供需侧数值。C面板中的供给侧数值清晰表明，BuiltWith样本中的价值主要来源于JavaScript代码——这一结果具有良好的合理性验证意义，因为正如前文所述，我们在该样本中以JavaScript包作为开源软件（OSS）的替代指标。第二高的数值由TypeScript产生，这令人安心，因为TypeScript是JavaScript的超集。D面板在使用侧呈现出相似模式：大部分价值仍来自JavaScript，而TypeScript则位列第二。其他编程语言对供需侧数值的贡献微乎其微。总体而言，这些发现与各类语言的主要应用场景（网页编程与应用编程）基本一致，也印证了这样一个观点：产生价值的语言未必与公众广泛使用的语言完全相同。

--- 图3 如图所示 ---

图3展示了基于BuiltWith在线数据，按 NAICS 两位数代码划分的各行业软件服务外包（OSS）需求侧价值。[20]这一数值可解读为各行业因OSS的存在而获得的价值。使用价值最高的行业是“专业、科学和技术服务”，约为430亿美元；“零售贸易”以及“行政与支持服务、废物管理和修复服务”分别贡献了360亿美元和350亿美元，共同构成了OSS需求侧对外部价值的重要组成部分。相比之下，“采矿、采石及油气开采”、“公用事业”以及“农业、林业、渔业和狩猎”等行业的贡献占比较小。后三类行业属于典型的非服务业，因此软件在这些领域的应用价值相对有限。

——此处的图4——

图3展示了供应端（图A）和需求端（图B）的洛伦兹曲线以及部分程序员所贡献的代码库数量。若洛伦兹曲线恰好位于45度线上，则意味着程序员的价值分布极为均匀。然而图A显示的洛伦兹曲线几乎呈平直状态，且最后那部分程序员贡献的比例急剧上升。这表明供应端的价值分布极不均衡，其集中度远超80/20法则的标准。事实上，最后5%的程序员（即3000名程序员）贡献了超过93%的供应端价值；同样地，图B在考虑使用率后显示，这最后5%的程序员贡献了超过96%的需求端价值。总体而言，这表明极少数程序员创造了企业用于编写自身代码的大部分开源软件代码。在图A和图B中，我们还可以看到：为贡献价值最高的10%-15%程序员所维护的代码库数量有所增加。这表明，少数程序员所产生的价值分布不均现象，并非仅源于少数高价值代码库的存在，而是由于这些贡献者对大量代码库作出了贡献所致。

5. 结论

[20] 由于人口普查数据包含客户的专有信息，因此未能完整呈现整个数据集中的各行业分布；因此，我们只能利用BuiltWith提供的公开数据来展示各行业的数值分布。我们通过将BuiltWith网站与Orbis和Compust-at数据集中的行业信息进行匹配，实现了这一目标。通过该流程，94.6%的BuiltWith网站成功匹配到了所属行业。对于涉及多个行业的企业（域名），我们取其数值平均值并按行业进行分布展示。

在本研究中，我们利用两个独特的数据集——OSS人口普查数据和BuiltWith数据——对全球广泛使用的开源软件价值进行了估算。我们不仅能够估算现有代码的供给端价值（例如，重新编写每个广泛使用的开源软件代码所需的成本），还能估算私营经济的需求端价值（例如，每家使用某开源软件的公司重新编写该软件所需的成本）。尽管我们并未重点关注开源软件的“长尾”部分，但这正是本研究的另一贡献：聚焦于广泛使用的开源软件，使我们能够更精确地理解开源软件创造的价值，而非仅测量所有开源软件的替换成本（后者会高估真实价值，因为许多开源项目并未用于生产代码）。然而，尽管我们基于大量使用数据强调了开源软件在社会中的巨大价值，但要完全识别全球范围内使用的全部开源软件并不现实；因此，我们的需求端估算结果很可能低估了其真实价值。

经使用情况调整后，我们发现当使用全球各地的程序员时，开源软件（OSS）的需求侧价值高达8.8万亿美元（具体数值存在波动，取决于是否仅雇佣来自低收入或高收入国家的程序员）。不同编程语言、代码面向内部（即用于开发待销售的产品）还是面向外部（即用于公司网站）等因素，都会显著影响其价值水平。排名前六的编程语言贡献了84%的需求侧价值。此外，不同行业之间也存在显著差异；最后，程序员个人对价值贡献的贡献程度也存在差异：超过95%的需求侧价值仅由5%的程序员创造，而这些程序员不仅参与了少数广泛使用的项目，其贡献的项目数量远超那些处于价值分布低端的程序员。

总体而言，我们的研究结果表明：尽管开源软件（OSS）对经济的贡献在直接测量中通常显示为零（因为其价格为零，且仅凭公开数据难以准确量化其数量），但其实际价值不容忽视。本研究不仅为未来对开源软件的研究奠定了基础，也为全面探讨信息技术及其对全球经济日益增长的影响提供了重要依据。

参考文献

Almeida，D. A. 、 Murphy，G. C. 、 Wilson，G. 和 Hoye，M.（2017年5月）。软件开发者是否理解开源许可证？收录于2017年IEEE/ACM第25届国际程序理解会议（IC-PC）论文集（第1–11页）。IEEE。

Andreessen, M. (2011).《为什么软件正在改变世界》.访问日期:2023年5月1日.来源:https://www.wsj.com/articles/SB10001424053111903480904576512250915629460

亚里士多德。（1981）。《政治学》第二卷第三章。T. A. 辛克莱译。企鹅出版社，伦敦。

Blind，K. 、 Bohm，M. 、 Grzegorzewska，P. 、 Katz，A. 、 Muto，S. 、 Patsch，S. 和 Schubert，T.（2021）。开源软件与硬件对欧盟经济技术独立性、竞争力及创新能力的影响。欧盟委员会编。

Blind, K., & Schubert, T. (2023).开源软件对GDP的影响及其与研发和专利的互补性:证据与政策启示.《技术转移杂志》,第1–26页.

Boehm, B. W. (1984).软件工程经济学.《IEEE软件工程汇刊》(第1卷),第4–21页.

Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D., & Steece, B. (2009).《 COCOMO II软件成本估算》.普伦蒂斯霍尔出版社.

Branstetter、Lee G.、 Matej Drev 和 Namho Kwon .（2019）。 “与时俱进：传统制造业中的软件驱动创新。” 《管理科学》第65卷第2期：541–558页。

Brynjolfsson, E. (1993).信息技术的生产率悖论.《ACM通讯》,36(12),66–77.

Brynjolfsson, E., & Hitt, L. (1996).悖论消失了吗?关于信息系统支出回报的公司层面证据.《管理科学》,42(4),541–558.

Brynjolfsson, E., Rock, D., & Syverson, C. (2018).人工智能与现代生产力悖论:期望与统计数据的冲突.载于《人工智能经济学:一项议程》(第23–57页).芝加哥大学出版社.

Burton，R. M. 、 Hakonsson，D. D. 、 Nickerson，J. 、 Puranam，P. 、 Workiewicz，M. 和 Zenger，T.（2017）。GitHub：探索无领导型与层级型组织形式之间的过渡形态。*《组织设计杂志》*，*6*，第1–19页。

Conti，A. 、 Peukert，C. 与 Roche，M. |(2023)。 “为投资者提升IT实力？开源与初创企业融资：来自GitHub的证据。” *哈佛商学院工作论文第22-001号*。

Cornes, R., & Sandler, T. (1996).《外部性, 公共物品与俱乐部物品理论》.剑桥大学出版社.

DeStefano，T. 与J. Timmis（2023）。《需求冲击与数据分析扩散》，工作论文。

Dushnitsky, G., & Stroube, B. K. (2021).低代码创业:Shopify与另一种增长路径.《商业风险投资洞察期刊》,第16卷,e00251.

Eisfeldt, A. L., & Papanikolaou, D. (2014).无形资本的价值与所有权.《美国经济评论》,104(5),189–194.

欧盟委员会。（2020）。《2020–2023年开源软件战略》。卢森堡：欧洲共同体官方出版物办公室。

第14028号行政命令。（2021年）。《关于加强国家网络安全的行政命令》。2021年5月。

Fackler，T.、 Hofmann，M. 与 Laurentsyeva，N.（2023）。《*挑战重力：远程团队生产力的驱动力是什么？*》（第427期）。CRC TRR 190：理性与竞争。

Greenstein, S., & Nagle, F.(2014).数字暗物质与Apache的经济贡献.*研究政策*,*43*(4),623−631.

GitHub（2022）。“Octoverse：开源软件现状。”检索于2023年11月3日。https://octoverse.github.com/2022/top-programming-languages。

Hanisch, M., Haeussler, C., Berreiter, S., & Apel, S.(2018年7月).开发者在Linux内核开发项目中从边缘角色晋升为核心成员.载于《*管理学院学报*》(2018年第1卷第1期,第14263页).纽约州布赖尔克利夫庄园10510:管理学院.

Hardin，G. (1968).《公地悲剧》.《科学》. 162(3859): 1243−1248.

Henkel, J. (2009).信息披露的倡导者——开源开发者在商业企业中的作用.《工业与企业变革》,18(3),435−471.

Jacks, J. (2022).开源软件对软件产业的冲击速度,已超过软件产业对整个世界的冲击速度.访问日期:2023年5月1日.来源:https://www.coss.community/cossc/open-source-is eating-software-faster-than-software-is-eating-the-world-3b01

Kim, D. Y. (2020).产品市场绩效与开放性:客户异质性的调节作用.载于《*管理学院学报*》(2020年第2020卷第1期,第21309页).纽约州布赖尔克利夫庄园10510:管理学院.

Koning, R., Hasan, S., & Chatterji, A. (2022).实验设计与启动表现:基于A/B测试的证据.《管理科学》,68(9),6434−6453.

Krishnan, M. S., Kriebel, C. H., Kekre, S., & Mukhopadhyay, T. (2000).软件产品生产率与质量的实证分析.《管理科学》,46(6),745−759.

Lerner, J., & Tirole, J. (2005).开源许可的范围.《法律, 经济学与组织学杂志》,21(1),20−56.

Lorenz，M. O.（1905）。“衡量财富浓度的方法”。《美国统计协会出版物》。*美国统计协会出版物*，第9卷，第70期。第9卷（总第70期）：209−219页。文献代码：1905 PAmSA...9..209L。doi：10.2307/2276207。 JSTOR 2276207。

Lifshitz-Assaf, H., & Nagle, F. (2021).数字经济基于开源技术运行.以下是保护它的方法.《哈佛商业评论》数字文章.https://hbr.org/2021/09/the-digital economy-runs-on-open-source-heres-how-to-protect-it。

Lloyd, W. F. (1833).关于人口控制的两场演讲:于1832年迈克尔马斯学期在牛津大学发表.作者:JH Parker.

Maracke, C. (2019).自由及开源软件与基于 FRAND 的专利许可:如何在标准必要专利与自由及开源软件之间寻求平衡.《世界知识产权杂志》,22(3-4),78-102.

Murciano-Goroff, R., Zhuo, R., & Greenstein, S. (2021). 隐性软件与隐性价值创造:基于服务器软件使用情况的案例分析.*Research Policy*,*50*(9),104333.

Musseau，J.，Meyers，J. S.，Sieniawski，G. P.，Thompson，C. A.，& German，D.（2022年5月）。开源软件是否正在吞噬全球软件市场？基于Java二进制文件测量专有软件中开源软件的比例。收录于*第19届国际软件代码库挖掘会议论文集*（第561−565页）。

Nagle, F. (2018).通过贡献学习:通过为众包公共产品做出贡献获得竞争优势.《组织科学》,29(4),569−587.

Nagle，F. (2019a). 开源软件与企业生产率. 《管理科学》，65(3)，1191–1215。

Nagle，Frank (2019b). 《政府技术政策、社会价值与国家竞争力》. 哈佛商学院工作论文，第19-103号，2019年3月。

Nagle，F.，Dana，J.，Hoffman，J.，Randazzo，S.，& Zhou，Y. (2022). 自由及开源软件第二次普查——应用库。*Linux基金会、哈佛创新科学实验室（LISH）与开源安全基金会（OpenSSF）。https://www.linuxfoundation.org/research/census-ii-of-free-and-open-source-software应用库。*

NIPA（2023）。经济分析局，NIPA表5.6.5。访问日期：2023年11月14日，来源：https://apps.bea.gov/iTable/？reqid=19&step=3&isuri=1&select_all_years=0&nipa_table_list=331&series=q&first_year=2013&last_year=2023&scale=-9.

威廉·D·诺德豪斯，2006年，《Nonmarket Acco 单位的 National Accounting 原则》，载于《*美国国民账户的新架构*》（戴尔·W·约根森、J·史蒂文·兰德菲尔德与威廉·D·诺德豪斯主编），芝加哥大学出版社。

埃莉诺·奥斯特罗姆（1990）。《治理公共资源：集体行动制度的演进》。剑桥：剑桥大学出版社。

Peters, R. H., & Taylor, L. A. (2017).无形资本与投资-收益关系.《金融经济学杂志》,123(2), 251–272.

Robbins，C.、Korkmaz，G.、Guci，L.、Calderon，J. B. S. 与 Kramer，B.（2021）。《美国及其他国家开源软件投资现状初探（2009–2019年）》。

Singh，Shivendu Pratap (2020)*《产品、平台与开放式创新：关于技术创新的三篇论文》*。匹兹堡大学博士论文（未出版）

Solow，R. (1987). "我们最好当心。" *《纽约时报书评》*，1987年7月，第36页。

Statista（2023）。Statista Software Worldwide，访问日期：2023年11月14日，来源：https://www.statista.com/outlook/tmo/software/worldwide#revenue，访问日期：2023年11月。

Synopsys（2023）。2023 OSSRA：开源趋势深度解析。访问日期：2023年5月1日。来源：https://www.synopsys.com/blogs/software-security/open-source-trends-ossra报告/

Tang, S., Wang, Z., & Tong, T.(2023). 开源贡献中的知识治理:守门人的作用.载于《*管理学院学报*》(2023年第2023卷第1期,第17622页).纽约州布赖尔克利夫庄园:管理学院.

Tozzi，C. (2016). 《开源历史：Linux为何成功？》*Channel Futures*，2016年8月。访问日期：2023年11月3日。https://www.channelfutures.com/open-source/open source-history-why-did-linux-succeed

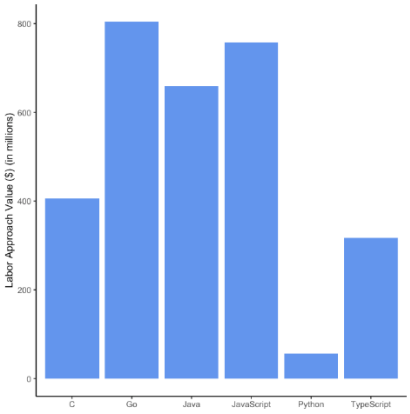
Wachs, J., Nitecki, M., Schueller, W., & Polleres, A. (2022).开源软件的地理分布:来自GitHub的证据.*技术预测与社会变革*,*176*,121478.

Williamson, S. (2006).《宏观经济理论笔记》.圣路易斯大学经济系.

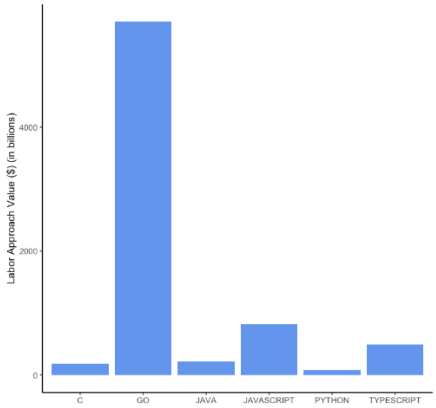
Zhang, Y., Zhou, M., Mockus, A., & Jin, Z.(2019).企业参与OpenStack开发——一项关于OpenStack的实证研究.《IEEE软件工程汇刊》,47(10),2242–2259.

图1开源软件的价值：主流编程语言

A组：供应方



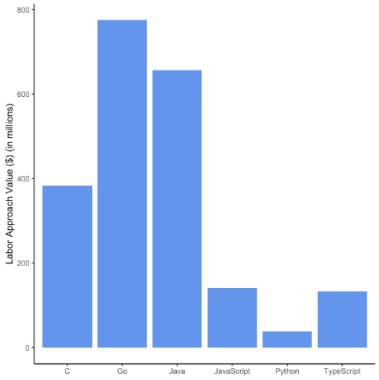
B面板：需求侧



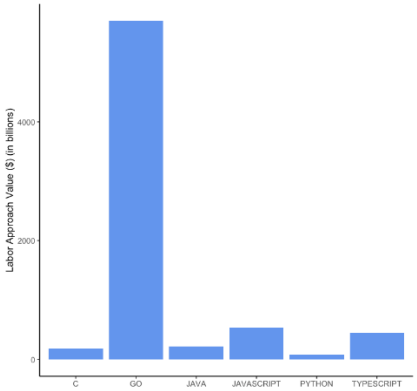
注。图表展示了根据GitHub及Go语言统计的前五大编程语言的劳动力市场价值。图A呈现供给端数据，图B则包含使用量数据。在劳动力方面，我们采用了方法论部分所述的全球程序员平均工资估算值。

图2：前五大编程语言（含Go）及各类数据源中的开源价值

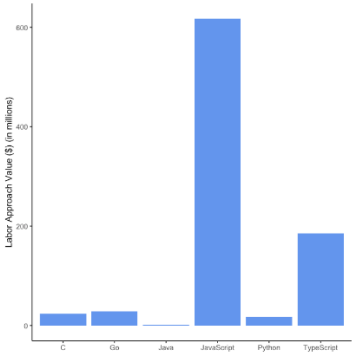
A组：人口普查。供给方面



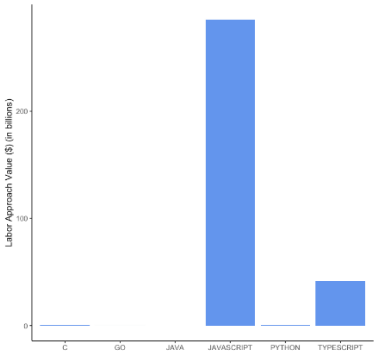
B面板：人口普查。需求侧



C面板：BuiltWith。供给侧

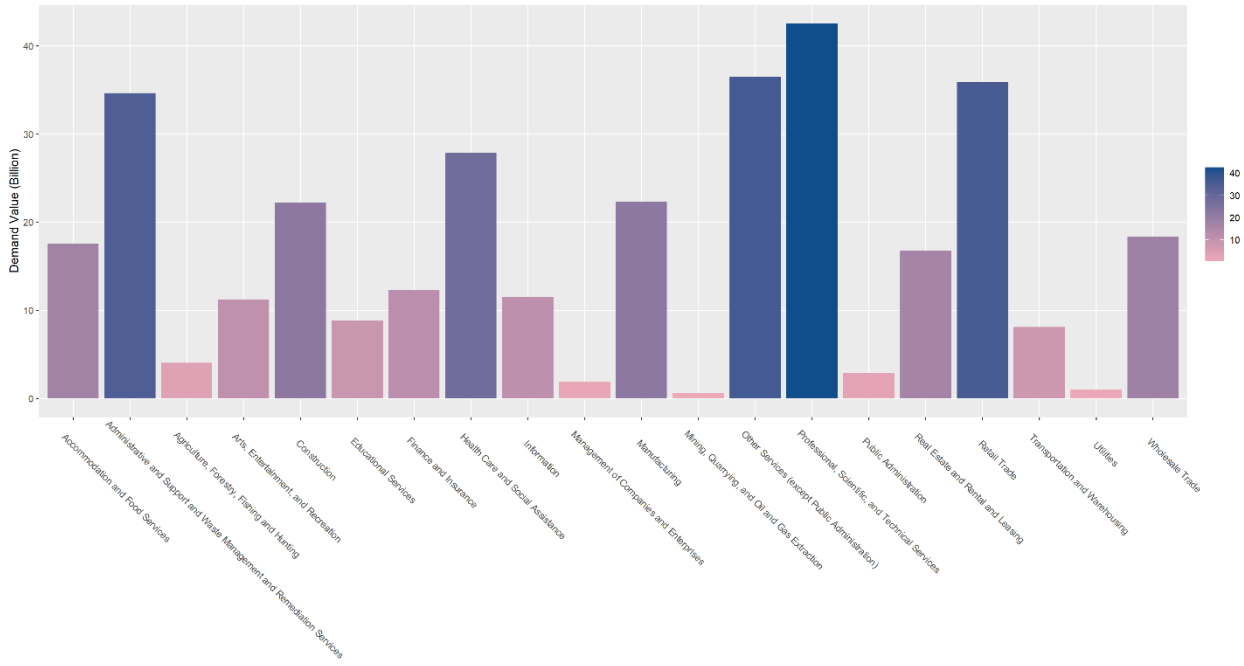


D面板：BuiltWith。需求侧



注：图表展示了排名前五的语言（根据 GitHub 数据）及 Go 语言的劳动力与商品市场价值，并按数据来源分为面向内向（Census）和面向外向（BuiltWith）两类。图 A 和图 B 分别展示了 Census 数据的供给侧与需求侧数值；图 C 和图 D 则展示了 BuiltWith 数据的供给侧与需求侧数值。

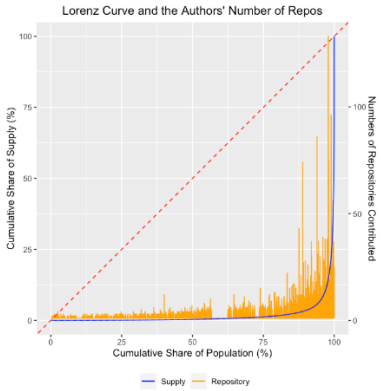
图3网站各行业开源价值



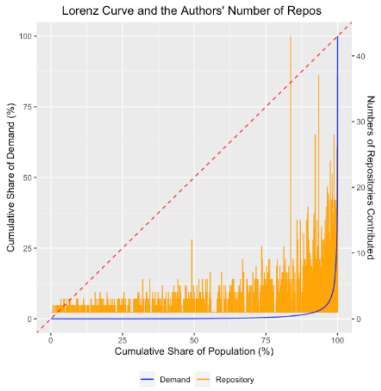
注。该图展示了基于Built With数据、涵盖 NAICS 两位代码行业的需求侧劳动力价值。对于涉及多个行业的企业（领域），我们取其平均值并按行业进行分配。

图4开源价值创造的分布情况

A组：供应方



B面板：需求侧



注。图表展示了每位开发者对劳动力市场价值的贡献（蓝色曲线）以及部分程序员贡献的代码仓库数量（黄色曲线）的洛伦兹曲线。图A展示供给端情况，图B则反映了使用情况。

表1代码行数及使用情况的描述性统计

	总数	平均；均值	SD	观察
A组：人口普查				
代码行数 – 所有软件包	261,653,728	142,203.1	887,937.2	1,840
代码语言排行榜：前五名语言及 Go 语言	189,673,184	113,712.9	702,832.1	1,668
使用说明 – 所有软件包	2,709,155	1472.4	2,167.9	1,840
使用情况——排名前五的语言及 Go 语言	2,497,785	1497.5	2,228.8	1,668
B组：BuiltWith				
代码行数 – 所有软件包	82,504,613	111,342.3	613,488.1	741
代码语言排行榜：前五名语言及 Go 语言	58,664,935	79,925.0	354,415.0	734
使用说明 – 所有软件包	142,794.4	192.7	733.4	741
使用情况——排名前五的语言及 Go 语言	142,751.2	194.5	736.6	734

注：这些统计数据基于不同代码库中的代码行。图A(B)展示了使用第1组所有软件包时，人口普查（BuiltWith）数据在不同代码行及使用场景下的总和、均值、标准差及观测值数量（详见表A1）。

表2开源的劳动力市场价值

	劳动力供应	劳动力需求
工资：较低	12.2亿美元	2.59万亿美元
工资：全球标准	41.5亿美元	8.80万亿美元；13.18万亿美元
工资：较高	62.2亿美元	

注：高薪情景基于美国程序员的平均薪资，低薪情景则基于2020年印度程序员的平均薪资。全球薪资数据取自表A4所列国家的平均薪资，并根据其对开源软件（OSS）的贡献程度进行加权。这些估算仅涵盖被归入第1类（见表A1）的软件所使用的编程语言。

在线附录

表 **A1**每个类别内的语言

类型	语言
A组：桶1 – 语言	
标记语言	BIBTEX
标记语言	COLDFUSION HTML
标记语言	DOCBOOK XML
标记语言	低增生性急性髓细胞性白血病
标记语言	HTML
标记语言	HXML
标记语言	JAVAEE XML
标记语言	标记语言
标记语言	梅森
标记语言	MXML
标记语言	RELAX-NG 紧凑型
标记语言	RHTML
标记语言	文本
标记语言	可扩展标记语言
标记语言	XQUERY
标记语言	YAML
编程语言	ABNF
编程语言	ACTIONSCRIPT
编程语言	ADA
编程语言	APPLESCRIPT
编程语言	ARDUINO
编程语言	ASPECTJ
编程语言	ASPX-CS
编程语言	ASPX-VB
编程语言	AWK
编程语言	C
编程语言	C#
编程语言	CHARMCI
编程语言	CLOJURE
编程语言	COFFEESCRIPT
编程语言	通用 LISP
编程语言	CSS
编程语言	CUDA
编程语言	CYTHON
编程语言	D
编程语言	DART
编程语言	德尔菲
编程语言	EASYTRIEVE
编程语言	EC
编程语言	灵药；药水
编程语言	ELM
编程语言	EMACSLISP
编程语言	ERLANG
编程语言	F#
编程语言	鱼类；鱼类生物
编程语言	福斯
编程语言	FORTRAN
编程语言	FORTRANFIXED

编程语言	差距；差距
编程语言	格金
编程语言	GLSL
编程语言	走
编程语言	GRAPHVIZ
编程语言	GROOVY
编程语言	HASKELL
编程语言	HAXE
编程语言	本地指令集
编程语言	Java
编程语言	JAVA 服务器页面
编程语言	JAVASCRIPT
编程语言	KOTLIN
编程语言	LESSCSS
编程语言	液体
编程语言	LIVESCRIPT
编程语言	LLVM
编程语言	标志
编程语言	LUA
编程语言	MATHEMATICA
编程语言	MINISCRIPT
编程语言	MODULA-2
编程语言	海军航空医学院
编程语言	NIX
编程语言	目标-C
编程语言	目标-J
编程语言	OCAML
编程语言	OPENEDGE ABL
编程语言	PAWN
编程语言	珍珠
编程语言	英文超文本预处理语言的缩写。是一种内嵌式的语言
编程语言	PL/PGSQL
编程语言	附言
编程语言	POVRAY
编程语言	序言
编程语言	属性
编程语言	木偶
编程语言	几何体
编程语言	REASONML
编程语言	REBOL
编程语言	REDCODE
编程语言	REXX
编程语言	RUBY
编程语言	锈迹；腐蚀
编程语言	S
编程语言	SASS
编程语言	比例尺
编程语言	一种"开放源码"软件
编程语言	SCSS
编程语言	纤细型；苗条型
编程语言	SMALLTALK
编程语言	稳健性
编程语言	标准 ML
编程语言	SWIFT
编程语言	SWIG
编程语言	TADS 3

编程	语言	TCL
编程	语言	蒂夫里特
编程	语言	TRANSACT-SQL
编程	语言	街顶
编程	语言	类型脚本
编程	语言	VB.NET
编程	语言	VBScript
编程	语言	VCL
编程	语言	VIML
编程	语言	Web IDL

B面板：桶2 – 辅助语言

转义符/编译器/解释器/宏	处理器	气体
转义符/编译器/解释器/宏	处理器	M4
转义符/编译器/解释器/宏	处理器	RAGEL 红宝石主持人
配置		CMAKE
配置		初始化设置文件的后缀名
配置		生成文件
配置		NGINX 配置文件
配置		非特异免疫抑制疗法
配置		SQUIDCONF
配置		特拉福姆
配置		TOML
格式		格罗夫
IDE		NETBEANS 项目
模板引擎		猎豹
模板引擎		GENSHI
模板引擎		PUG
模板引擎		聪明的
模板引擎		速度
终端/批次		ANT
终端/批次		APACHECONF
终端/批次		XSLX
终端/批次		BATCHFILE
终端/批次		女专家；女权威
终端/批次		POWERSHELL
终端/批次		RPMSPEC
终端/批次		奇点
终端/批次		TCSH
翻译的；转化的		GETTEXT 目录

C面板：桶3 – 数据

数据	BNF
数据	微分
数据	DTD
数据	电子邮件
数据	JSON
数据	协议缓冲区
数据	RESTRUCTUREDTEXT
数据	纯文本
数据	可扩展样式表语言变换

表 A2全球工资最高的前 30 个国家

国家
美国、中国、
德国、印度
英国 巴西 俄罗
斯 法国 加拿大
日本 韩国 荷兰
西班牙 波兰 澳
大利亚 瑞典 乌
克兰 意大利 瑞
士 印度尼西亚
台湾 哥伦比亚
阿根廷 墨西哥
挪威 比利时 丹
麦 □

注：排名前30的国家按GitHub用户分享量升序排列，涵盖了2020年GitHub活动总量的88%。

表 A3使用第1和第2类别语言的开源项目在劳动力市场的价值

	劳动力供应	劳动力需求
工资：较低	12.3亿美元	2.6万亿美元
工资：全球标准	41.8亿美元	8.84万亿美元；
工资：较高	62.6亿美元	13.24万亿美元

注：高薪情景基于美国程序员的平均薪资，低薪情景则基于2020年印度程序员的平均薪资。全球薪资数据取自表A4所列国家的平均薪资水平。相关估算仅涵盖第1和第2组语言（详见表A1）。

表 A4使用第 1、2 和 3 组语言的开源在劳动力市场中的价值

	劳动力供应	劳动力需求
工资：较低	18.8亿美元	3.52万亿美元；11.96万亿美元
工资：全球标准	64.1亿美元	
工资：较高	95.9亿美元	17.91万亿美元

注：高薪情景基于美国程序员的平均薪资，低薪情景则基于2020年印度程序员的平均薪资；全球薪资数据取自表A4所列国家的平均薪资水平。相关估算仅涵盖第1、2、3类编程语言（详见表A1）。

表 A5商品篮子——等效开源与专有软件

开源软件	专有软件
Apache HTTP服务器	Windows Server 2008
大胆；自信	Adobe Audition
掺和器	Autodesk Maya
Elasticsearch	亚马逊肯德拉
FileZilla	SmartFTP
免费CAD	著名的三维辅助设计软件
GIMP	Adobe Photoshop
GNU Octave	矩阵实验室
GnuCash	QuickBooks
KeePass	1Password
LibreOffice	Microsoft Office 套件
MariaDB服务器	Microsoft SQL Server
以基为起点	Tableau
MySQL	Oracle MySQL
OpenVPN	ExpressVPN
PSPP	SPSS
Redis	Redis 企业版
张量流	TensorFlow Enterprise
虚拟盒	虚拟机
VLC 媒体播放器	CyberLink PowerDVD

表 A6使用第 1、2 和 3 组语言的开源产品市场价值

	商品需求桶 1	商品需求桶 1-2	商品需求，桶 1-3
工资：较低	1.77亿美元	1.79亿美元	2.42亿美元
工资：全球标准	1.77亿美元	1.79亿美元	2.42亿美元
工资：较高	1.77亿美元	1.79亿美元	2.42亿美元

注：高薪情景基于美国程序员的平均薪资，低薪情景则基于2020年印度程序员的平均薪资；全球薪资数据取自表A4所列国家的平均薪资水平。相关估算仅涵盖第1、2、3类编程语言（详见表A1）。

附录A）商品市场估值法

作为一种替代性估算方法，我们并未采用劳动力替代成本法，而是采用了商品替代价值法。我们筛选出若干具有类似闭源付费替代方案的开源软件包，并计算了若所有免费开源软件的商业用户都必须用付费替代方案替换这些软件所产生的成本（该计算方式类似于Greenstein 和 Nagle（2014）以及Murciano-Goroffet等人（2021）对网络服务器所做的测算）。随后，我们可以将这一替代性的p值与前述q值相结合，从而估算出开源软件的商品替代价值。该方法借鉴了Nordhaus（2006，第146页）的观点："……市场与非市场商品及服务的价格应基于可比的市场商品及服务进行推算。" 我们预计这两种方法得出的估算结果不会相近。恰恰相反，由于后一种商品市场法假设商品售价固定且可多次出售，而该固定价格通常低于通过劳动力成本重新开发所有软件包所估算的总价值，因此两种方法得出的价值存在显著差异。这些估算值之间的差异，本质上是由于某家公司介入，重新开发了缺失的OSS软件包，并以一定价格出售，而非所有公司都必须自行从头开始重新开发这些软件包。

采用商品市场方法时，我们的思想实验仍然是：我们生活在一个开源软件（OSS）并不存在的世界中，但必须通过一家企业来重新创造它——该公司会对目前免费提供的产品收取费用。为了通过商品市场方法评估OSS的价值，我们构建了一个由等效替代性专有软件组成的商品篮子，这些软件在公开市场上定价，作为OSS产品的替代品。该方法与先前文献（Greenstein and Nagle，2014； Murciano-Goroffet al.，2021）所采用的方法一致，不过这两项研究仅使用单一产品而非商品篮子，因为它们仅聚焦于一种类型的OSS（Web服务器）。由于目前缺乏现成的OSS专有替代品数据库，我们根据对OSS流行度的主观判断进行搜索，进而寻找相应的付费闭源替代品。最终形成的包含20个OSS软件包及其专有替代品的商品篮子，很好地反映了OSS的多样性。这些软件涵盖从媒体与设计软件、统计分析程序到数据库管理和Web服务器软件等多个领域。[21]

[21] 表A5展示了我们使用的开源软件（OSS）及其专有软件对应产品的组合。在构建该组合时，我们筛选了那些拥有功能与特性集高度相似的OSS对应产品的专有软件，并力求找出能够综合反映各类OSS广泛且代表性特征的软件组合。

基于我们构建的开源软件等效替代专有软件商品篮子，我们获得了每种专有软件等效物的价格，并计算了篮子中每个开源软件产品的 COCOMO 劳动力市场供给侧价值——由于缺乏专有软件的代码，该数值被用作专有软件 COCOMO 劳动力供给侧价值的代理指标（例如，支付程序员从零开始编写该专有软件所需的成本）。随后，我们计算了该商品篮子的平均 COCOMO 劳动力供给侧价值$V_{S\ Basket}$以及篮子的平均价格$P_{S\ Basket}$。[22]由于我们已知所有开源软件的劳动力市场供给侧价值$V_S{}^{Labor}$，可通过以下方程并借助简单的缩放变换，得出所有开源软件在商品市场上的价格P_S：

$$\frac{P_S}{V_S^{Labor}} = \frac{P_{S,Basket}}{V_{S,Basket}}$$

$$P_S = \frac{V_S^{Labor}}{V_{S,Basket}} P_{S,Basket} \tag{4}$$

由此得出以下商品市场供给侧价值：

$$V_S^{商品} = P_S * 1。 \tag{5}$$

随后，我们可以按以下方式计算出等效的需求侧商品市场价值：

$$V_D^{Goods} = P_S * Q. \tag{6}$$

从商品市场角度而言，我们考虑的是：如果一家企业生产所有现有的广泛使用的开源软件（OSS），并将其作为产品销售给客户，其收取的等效价格。

表A6第3列显示，无论程序员的来源国如何，各收入区间的需求侧价值均介于1.77亿美元至2.44亿美元之间。显然，商品市场的价值远低于劳动力需求价值——因为这家虚拟企业只需一次生产软件并批量销售给客户，就能以相对较低的价格实现盈利。此外，尽管 Greenstein 和 Nagle（2014）以及 Murciano-Goroff 等人（2021）仅聚焦于某一类软件（Web服务器），我们试图将其研究方法扩展至多种商品。然而，在此背景下，数据限制成为更主要的制约因素，使得研究本身更为困难。

[22] 我们通过采用软件3年使用寿命（即1/(1-0.66)）计算了商品市场专有篮子的平均价格，因此折旧系数为0.33。该结果符合美国国税局（IRS）关于软件折旧的规定："若可对计算机软件成本进行折旧，则应在36个月的使用年限内采用直线法。" https://www.irs.gov/publications/p946#en_US_2022_publink1000107354。

通过这种方法来估算价值时，需要做出远为繁多的假设。与纯粹的商品市场方法不同，该方法需要借助劳动力市场方法来调整估算规模，这最终会导致对开源软件价值的严重低估。此外，专有软件对应产品的定价策略对市场需求极为敏感。当商品市场方法应用于多种产品时，其隐含的核心假设是：我们的软件组合与样本开源软件的市场需求相似，因此它们应具有相应的价格水平。

另一种更为简化的商品市场粗略估算方法——该方法不考虑代码行数，也不依赖劳动力市场方法中的规模效应——只需将参考商品的价格乘以使用量即可。我们可以采用参考价格向量 p = (69.99,1610.17,5800) 中记录的专有商品篮子的最低价、均价和最高价，将其与综合人口普查数据和 BuiltWith 样本（表 1）中的使用量相乘。基于这些估算的专有商品价格假设，商品市场需求价值范围将在 0.2 万亿美元至 16.5 万亿美元之间（最低价–最高价），平均价格对应的估值约为 4.5 万亿美元。然而，尽管该估算方法比劳动力市场方法产生略高的方差，但其本质存在缺陷：它简单地假设所有开源产品的估算价格均基于参考价格完全一致。我们进一步认为，这种非常简单的粗略估算实际上是一种“橙子与苹果”的类比：因为我们购物篮中的商品都是功能完备的独立软件包，而Census和BuiltWith数据集中的软件包则由应用程序库组成，而这些应用程序库通常比独立软件包更小。

鉴于在采用商品市场法时因数据不足而必须做出诸多复杂假设，我们更侧重于正文重点阐述的劳动力成本法；但为确保内容完整，此处亦一并纳入该方法。

文献主要观点：这篇哈佛商学院等机构的研究指出，开源软件作为数字经济核心基础设施，因免费属性与使用难以追踪，其真实价值长期未被传统经济指标衡量。研究依托全球企业开源使用数据，从供给与需求两侧测算其价值。

结果显示，广泛使用的开源软件**供给侧重建成本仅 41.5 亿美元**，而**需求侧使用价值高达 8.8 万亿美元**，二者差距悬殊。若没有开源软件，企业软件支出将增至当前的 3.5 倍，数字经济运行成本大幅上升。

开源价值分布高度集中：5% 的开发者贡献 96% 的需求侧价值，六大编程语言创造 84% 的需求价值，专业服务、零售等行业受益最为突出。

该研究填补了开源价值测算空白，证实开源是极具影响力的全球公共品，也为企业参与开源维护、政策层面加大生态支持提供了重要依据。

开源软件的价值研究核心观点：

这份研究聚焦开源软件的真实经济价值，指出其作为数字经济关键公共品，因免费、使用难追踪的特性，长期被传统经济指标低估。研究依托企业级开源使用数据，从供给与需求两侧开展测算。

结果显示，广泛使用的开源软件供给侧重建成本约 41.5 亿美元，而需求侧使用价值高达 8.8 万亿美元，若无开源软件，企业软件支出将达到当前的 3.5 倍。价值分布呈现高度集中特征，仅 5% 的开发者贡献 96% 的需求侧价值，六大编程语言创造 84% 的需求价值，专业服务、零售等行业受益最显著。

该研究弥补了开源价值量化的空白，证实开源对经济的巨大贡献，同时为企业参与开源维护、政策制定者完善生态支持提供了关键数据支撑。